RIVR – Texas Hold'em
Iteration Planning, Execution and Testing

CS 482, Team 23
Chris, Finn, Kelechi and Royal

November 3, 2026

About this report. RIVR is our team's Texas Hold'em web application. This document explains what we planned for Iteration 2, what we worked on, how we tested it, and what we want to improve next time. We also keep our Iteration 1 plan for reference. Planned stories are separate from completed stories, and a feature is not considered accepted until the professor approves it.

The Iteration 2 assignment lists November 5, 2026 as the due date, with a grace period to Sunday night. We still need to check the exact deadline on Moodle. Our planning email was reportedly sent on October 27, 2026 at 7:47 PM. The live demo is separate from the delivery deadline.

Contents

1 System Design **2**

2 Main Iteration Planning Table **3**

3 Iteration 2: Planning and Execution **3**

4 Testing and Coverage Report **4**

5 Process, Traceability and Retrospective **7**

6 Release and Demo Checklist **7**

A Actual work hours worksheet **8**

1 System Design

RIVR uses three main parts: a React/TypeScript frontend, a Django backend, and a PostgreSQL database. The backend handles important checks, such as who can change data and when XP or chips can be awarded. This keeps the browser from changing saved values on its own.

1.1 Diagrams

Our sequence diagram shows how a multiplayer poker hand could work. A player sends an action, the server checks it, the database saves the result, and the other players receive the updated game state. It also shows how XP could be awarded after a hand. Some of these gameplay features are still planned, so the diagram represents the intended design, not a finished multiplayer game.

1.1.1 Proposed Multiplayer Gameplay Sequence

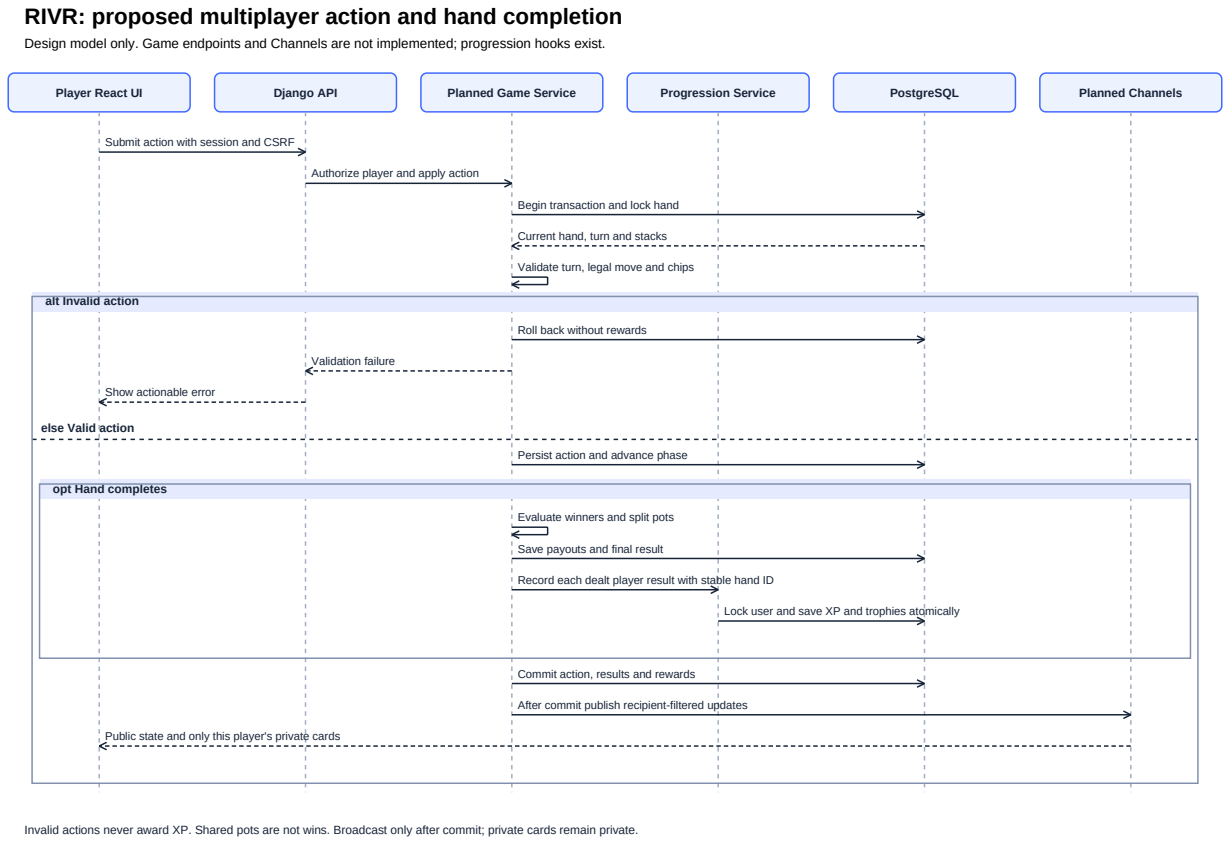


Figure 1: Proposed multiplayer gameplay sequence showing player actions, server checks, saved results, and game updates.

2 Main Iteration Planning Table

Iteration	Owner	Planned stories	Points
1	Chris	S13 public table; S10 browse tables	4
1	Finn	S2 registration; S1 login/logout; S4 guests	4
1	Kelechi	S23 admin cleanup; S35 themes	4
1	Royal	S5 image upload; S14 starting chips; S29 daily chips	4
1	Team	Updated team plan	16
2	Chris	S32 XP/achievements (3); S18 leaderboard (1)	4
2	Finn	S19 add/remove friends	5
2	Kelechi	S16 chip balance (2); S15 buy-in (1); S17 stats (1)	4
2	Royal	S11 filters (1); S27 private table (1); S26 spectate (2)	4
2	Team	Reported planning-email content	17

These are the points assigned in our plans, not the points the professor has accepted. The team planned 17 points for Iteration 2. Our completed work and hours are recorded below. We still need to confirm the final totals before submitting. Some table filters and chip features started in Iteration 1, so the Iteration 2 report should only count new work.

3 Iteration 2: Planning and Execution

3.1 Team branches and contributions

We worked on separate Git branches so we could build our features without changing each other's code. Chris's work is on `saia`. Finn's `feature/S19-friends` branch has commits. Kelechi branches are also recorded on his branch. Royal has not created a branch yet or pushed any code. The branch records are saved in `docs/quality/branch-evidence.json`.

3.2 Chris: S32 and S18

S32, Earn XP and achievements (3 planned points, issue #45). Chris built an XP and achievement system with 24 trophies. Players can earn XP for actions that are already connected to the app, including uploading an avatar, creating a public table, and changing their theme. The code also includes rewards for winning hands and tournaments, but those still need to be connected to the future gameplay features. One-time rewards cannot be collected twice.

S18, See the leaderboard (1 planned point, issue #32). Chris built an all-time leaderboard using XP saved in the database. It shows players' ranks, usernames, avatars, and XP. It also handles players tied for the same rank and lets a signed-in player see their own rank. The leaderboard and XP behavior have tests, but the team still needs to check both features again after merging everyone's work.

3.3 Finn: S19 - Add and Remove Friends

S19, 5 planned points. Finn worked on the friends feature. The goal is for players to send friend requests, accept or decline requests, see their friends, and remove friends.

3.4 Kelechi: S16, S15 and S17 - Player Data and Chips

S16, Save chip balance (2 planned points). Kelechi's task is to save a player's chip balance so it stays correct when they leave and return to the app.

S15, Check table buy-in (1 planned point). This story checks whether a player has enough chips to join a table. The check should happen in the backend, not just on the screen.

S17, See my stats (1 planned point). This story lets players view their own statistics.

3.5 Royal: S11, S27 and S26 - Lobby and Table Features

S11, Filter tables (1 planned point). Royal's task is to improve how players filter the lobby to find tables. Because the lobby already had some filters.

S27, Make a private table (1 planned point). This story allows a player to create a table that is not listed with the public tables.

S26, Watch a game (2 planned points). This story lets someone watch a poker game without playing. Viewers should not be able to make moves or see another player's private cards.

3.6 Iteration 2 execution table

Story	Owner	Work completed / status	Hours	Points done
S32	Chris	XP and trophies built; gameplay connections incomplete	2	2
S18	Chris	Leaderboard built; final merge check needed	0.5	2
S19	Finn	Friends branch has commits; working features to confirm	2.5	5
S16	Kelechi	Chip-balance work to confirm	1	2
S15	Kelechi	Buy-in check to confirm	0.5	1
S17	Kelechi	Stats feature to confirm	0.5	1
S11	Royal	New lobby filters to confirm	TBD	2
S27	Royal	Private-table feature to confirm	TBD	1
S26	Royal	Spectator feature to confirm	TBD	1
Team total (17 points planned)			**TBD**	**TBD**

4 Testing and Coverage Report

Measurement time (UTC): `2026-11-03T20:54:42.191515+00:00`.

Scope	Files	Statements	Branches
Backend	37	881/900 (97.89%)	203/210 (96.67%)
Frontend	33	541/561 (96.43%)	453/489 (92.63%)

JUnit evidence: backend 363 tests; frontend 115 tests. Failures/errors: 0 on each side.

Equal-weight means: statements 97.16%; branches 94.65%.

The manifest and raw reports provide exact denominators, missing code and exclusions.

These coverage numbers come from the saved reports for the source commit listed above. Both the frontend and backend results are above the assignment's targets of more than 80% code coverage and more than 50% branch coverage for an A. We must run the tests again after merging the whole

```
christophersaia@Christophers-MacBook-Air frontend % npm run test:coverage

lavigate to Command
croll to Previous Command (⌘↑)
croll to Next Command (⌘↓)

> v8
```

	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	96.43	92.63	97.23	98.21	
src	88.46	81.25	88.88	86.95	
App.tsx	95.83	81.25	88.88	95.23	146
main.tsx	0	100	100	0	7-9
src/api	100	100	100	100	
client.ts	100	100	100	100	
src/components	100	100	100	100	
AppLayout.tsx	100	100	100	100	
FormField.tsx	100	100	100	100	
src/features/achievements	100	95	100	100	
AchievementsPage.tsx	100	95	100	100	36
src/features/auth	98.59	96.25	100	100	
AuthLayout.tsx	100	100	100	100	

Figure 2: Full backend per-file coverage generated from the tool's raw JSON.

team's work, then update this section if the results change. Coverage measures how much code our tests run; it does not prove that every user story is finished.

4.1 Tool-generated report prints

```
Backend full-application coverage
Measured runtime files (including unexecuted files)
Source commit: 1af0df00c370
File                                                                    Statements      Branches
-------------------------------------------------------------------------------------------------
accounts/__init__.py                                                       0/0           0/0
accounts/admin.py                                                          4/4           0/0
accounts/models.py                                                       11/11           0/0
accounts/serializers.py                                                  43/43           8/8
accounts/services.py                                                     24/24           6/6
accounts/urls.py                                                           4/4           0/0
accounts/views.py                                                        68/68           6/6
config/__init__.py                                                         0/0           0/0
config/asgi.py                                                             0/4           0/0
config/settings.py                                                       31/32           1/2
config/urls.py                                                           10/10           1/2
config/wsgi.py                                                             0/4           0/0
conftest.py                                                              35/35           0/0
core/__init__.py                                                           0/0           0/0
core/views.py                                                            15/15           0/0
manage.py                                                                  0/8           0/2
profiles/__init__.py                                                       0/0           0/0
profiles/avatars.py                                                      59/59           9/10
profiles/rewards.py                                                      42/42         10/10
profiles/urls.py                                                           4/4           0/0
profiles/views.py                                                        52/52           6/6
progression/__init__.py                                                    0/0           0/0
progression/catalog.py                                                     5/5           0/0
progression/facts.py                                                     60/60         50/50
progression/leaderboard.py                                               51/51         12/12
progression/models.py                                                    22/22           0/0
progression/services.py                                                  71/73         40/42
progression/views.py                                                     18/18           0/0
tables/__init__.py                                                         0/0           0/0
tables/admin.py                                                          26/26           4/4
tables/cleanup.py                                                        35/35           8/8
tables/models.py                                                         48/48         12/12
tables/pagination.py                                                     11/11           0/0
tables/permissions.py                                                      8/8           0/0
tables/serializers.py                                                    58/58         14/14
tables/urls.py                                                             4/4           0/0
tables/views.py                                                          62/62         16/16
-------------------------------------------------------------------------------------------------
TOTAL                                                                  97.89%        96.67%
Tests, generated migrations and type-only declarations are excluded.
Backend includes config and launchers; frontend includes main.tsx.
See manifest.json and raw tool JSON for denominators and missing code.
```

Figure 3: Full backend per-file coverage generated from the tool's raw JSON.

```
Frontend full-application coverage
Measured runtime files (including unexecuted files)
Source commit: 1af0df00c370
```

File	Statements	Branches
src/App.tsx	23/24	13/16
src/api/client.ts	11/11	4/4
src/components/AppLayout.tsx	10/10	41/41
src/components/FormField.tsx	2/2	10/10
src/features/achievements/AchievementsPage.tsx	27/27	19/20
src/features/auth/AuthLayout.tsx	10/10	12/12
src/features/auth/GuestPage.tsx	9/9	2/2
src/features/auth/RegisterPage.tsx	14/14	6/6
src/features/auth/SessionGate.tsx	11/11	12/12
src/features/auth/SignInPage.tsx	9/9	2/2
src/features/auth/api.ts	12/12	0/0
src/features/auth/types.ts	0/0	0/0
src/features/auth/useAuthForm.ts	46/48	19/22
src/features/auth/validation.ts	29/29	24/24
src/features/leaderboard/LeaderboardPage.tsx	28/30	35/37
src/features/profile/AccountCard.tsx	34/41	31/41
src/features/profile/ProfileGate.tsx	8/8	10/10
src/features/profile/ProfileProvider.tsx	24/24	25/27
src/features/profile/RewardsPage.tsx	31/34	28/32
src/features/profile/api.ts	8/8	0/0
src/features/profile/context.ts	3/3	0/0
src/features/profile/types.ts	0/0	0/0
src/features/tables/CreateTablePage.tsx	52/52	48/52
src/features/tables/LobbyPage.tsx	61/63	58/62
src/features/tables/TableDetailPage.tsx	14/14	7/9
src/features/tables/api.ts	8/8	0/0
src/features/tables/types.ts	0/0	0/0
src/features/tables/validation.ts	20/20	28/28
src/features/themes/SettingsPage.tsx	18/18	16/16
src/features/themes/ThemeProvider.tsx	10/10	0/0
src/features/themes/theme.ts	5/5	2/2
src/features/themes/useTheme.ts	4/5	1/2
src/main.tsx	0/2	0/0
TOTAL	96.43%	92.63%

```
Tests, generated migrations and type-only declarations are excluded.
Backend includes config and launchers; frontend includes main.tsx.
See manifest.json and raw tool JSON for denominators and missing code.
```

Figure 4: Full frontend per-file coverage generated from the tool's raw JSON.

4.2 Adequacy, gaps and improvement

Our tests cover important cases such as saving data, rejecting invalid requests, checking permissions, avoiding repeated XP rewards, and showing the correct leaderboard ranks. Frontend tests also check forms, errors, and loading states. The current coverage results are strong, but we still need to test the final version after everyone's code is combined.

The main testing gap is that some features depend on parts of the poker game that are not finished yet. For the next iteration, we should test each new feature on its own and then run a few full user flows with the real database before the deadline.

5 Process, Traceability and Retrospective

Our GitHub Projects board uses Backlog, Ready, In progress, In review, Acceptance Test, and Done. Stories should stay in Acceptance Test until Dr. Rocha approves them. We use story IDs and GitHub issue numbers in our commits when possible, including #45 and #32 for Chris's Iteration 2 work.

Our team planned 17 story points. The actual number completed is still being checked, and the final number accepted will depend on the demo. We also need real board timestamps before we can give an accurate cycle time. We should not guess these numbers from Git commits.

For the next iteration, we want to communicate earlier about features that depend on each other, merge our branches sooner, and test the app together instead of waiting until the end. We also need to keep better track of hours and update the board as we work. That will make the next report easier to write and give us more time to fix problems before the demo.

6 Release and Demo Checklist

1. Compare the reported October 27, 7:47 PM email time against the assignment release date; enter actual hours and confirm Moodle/demo dates.
2. Push original feature branches; peer-review and integrate without losing authorship. Main must contain the release before cutoff.
3. Run the full quality script, export truthful evidence for the source SHA, build this report and commit the PDF/source/images.
4. Rehearse S32/S18 real actions and failure cases. Show the proposed gameplay model separately from running features.
5. Every member presents their own work and can explain scope, tests and commits. Communicate a real absence in advance.
6. Record CTO decisions after the demo; accepted stories may move to Done and rejected work returns to Backlog.

A Actual work hours worksheet

Enter actual hours, or honest retrospective estimates explicitly labelled as estimates, for each story. These are editable placeholders, not invented records. Confirm each contributor's work with that person; planned ownership and later repair work are distinct.

Iteration	Story	Planned owner	Contributor evidence	Actual hours
1	S13	Chris	Chris	1.5
1	S10	Chris	Chris	.5
1	S2	Finn	Finn	.5
1	S1	Finn	Finn	.5
1	S4	Finn	Finn	.5
1	S23	Kelechi	Kelechi	.5
1	S35	Kelechi	Kelechi	1
1	S5	Royal	Chris	NA
1	S14	Royal	Chris	NA
1	S29	Royal	Chris	NA
2	S32	Chris	Chris	2
2	S18	Chris	Chris	.5
2	S19	Finn	Finn	2.5
2	S16	Kelechi	Kelechi	1
2	S15	Kelechi	Kelechi	.5
2	S17	Kelechi	Kelechi	.5
2	S11	Royal	TBD	TBD
2	S27	Royal	TBD	TBD
2	S26	Royal	TBD	TBD